

Smart Railway Maintenance Orchestrator — Task Summary

1. Project Goal

Build a web-based AI-powered **Railway Maintenance Orchestrator** for Indian Railways.

The system should NOT replace existing railway systems such as TMS, SMMS, TDMS and COA.

Instead, it acts as an intelligent coordination layer that uses their data to:

- Identify maintenance requirements
- Calculate maintenance risk and priority
- Find suitable maintenance opportunities
- Combine work from multiple departments
- Consider train traffic and expected disruption
- Consider crew, machine and material availability
- Generate weekly and monthly maintenance plans
- Simulate "what-if" scenarios
- Automatically re-plan when conditions change

The key idea is:

Do not simply schedule maintenance blocks. Find the maximum useful maintenance that can safely be completed during every available railway operating window.

2. Main Innovation

The system should be based around the concept of a:

"Maintenance Opportunity"

Instead of:

Maintenance Request → Block

use:

Railway Data → Available Opportunity → Maintenance Bundle → Optimized Plan

Example:

A section has a 2-hour low-traffic window.

Available work:

- Track repair: 90 minutes
- Signal inspection: 45 minutes
- OHE inspection: 60 minutes

The system identifies that all three can be performed in the same section/window.

It creates:

One Maintenance Opportunity

- Time: 02:00–04:00
- Section: A-B
- Engineering: Track repair
- S&T: Signal inspection
- Electrical: OHE inspection

This reduces the need for separate blocks and reduces disruption to train operations.

3. Users

Create role-based access for:

Railway Planner

Can:

- View recommended plans
- Approve/reject plans
- Run simulations
- Modify schedules
- View conflicts

Engineering Department

Can:

- View track maintenance
- Update task status
- Add/update resource availability

S&T Department

Can:

- View signal/telecom tasks
- Update task status

Electrical/Traction Department

Can:

- View OHE/traction tasks
- Update task status

Admin

Can:

- Manage users
- Manage sections
- Manage trains
- Manage resources
- Configure system parameters

For the prototype, authentication can be simplified/mock authentication.

4. Data Sources

Create realistic synthetic data because real Indian Railway operational data will not be available.

Simulate data coming from:

TMS

Track maintenance issues.

Fields:

- task_id
- section_id
- location/km
- defect_type
- severity
- detected_date
- due_date
- estimated_duration

- risk_score
- status

SMMS

Signal & Telecom issues.

Fields:

- task_id
- section_id
- location
- equipment_type
- defect_type
- severity
- estimated_duration
- risk_score
- status

TDMS

Traction/OHE issues.

Fields:

- task_id
- section_id
- location
- equipment_type
- defect_type
- severity
- estimated_duration
- risk_score
- status

COA / Train Data

Simulate:

- train_number
- train_name
- train_type
- section_id
- arrival_time
- departure_time
- direction
- expected_delay
- traffic_level

Include passenger and goods trains.

Resource Data

Simulate:

- department
 - crew_id
 - crew_type
 - available_time
 - machine
 - machine_type
 - material availability
 - working hours
-

5. Railway Network Model

Create a demo railway corridor containing:

- Stations
- Railway sections
- Tracks
- Signals
- OHE assets

Example:

Station A → Section A1 → Station B → Section B1 → Station C → Section C1 → Station D

Each section should have:

- Track health
- OHE health
- Signal health
- Traffic density
- Current maintenance tasks
- Available maintenance windows

Use a graph/network representation internally.

6. Risk / Priority Engine

Create an explainable risk scoring engine.

Each maintenance task should receive a score between 0–100.

Possible factors:

- Severity
- Age of defect
- Safety impact
- Traffic density
- Route importance
- Asset criticality
- Weather impact
- Previous failures
- Proximity to other defects
- Due date

Example:

```
Risk Score =  
  
Severity × 30%  
+ Asset Criticality × 20%  
+ Age/Urgency × 15%  
+ Traffic Impact × 15%  
+ Safety Impact × 20%
```

Do NOT make the system a black box.

Every recommendation should explain WHY the task received its priority.

Example:

"Priority 94 because severity is high, defect is overdue by 4 days, section has high traffic density and asset is safety-critical."

ML can be added as an optional prediction/ranking layer, but the prototype must remain explainable.

7. Maintenance Opportunity Engine

This is the main feature.

The system should search upcoming time windows and identify:

- Low traffic periods
- No conflicting trains

- Available crews
- Available machines
- Available materials
- Nearby maintenance tasks
- Compatible department work

For every possible window, calculate:

Maintenance Opportunity Score

Consider:

- Number of tasks that can be completed
- Total maintenance value
- Task priority
- Number of departments coordinated
- Train disruption
- Resource availability
- Safety constraints
- Time available

Example output:

Opportunity #17

Section: A-B

Time: 02:00-04:00

Tasks:

- ✓ Track repair
- ✓ Signal inspection
- ✓ OHE maintenance
- ✓ Track geometry inspection

Departments:

Engineering + S&T + Electrical

Expected train impact: 6 minutes

Maintenance value: High

Opportunity Score: 93/100

Recommendation: ACCEPT

8. Smart Maintenance Bundling

Find tasks that can be performed together.

Tasks can be bundled when:

- Same section
- Nearby location
- Compatible work
- Same/similar time window
- Required resources available
- No safety conflict

Do not simply combine every task.

The system must check compatibility.

Example:

Section A

|

|— Track repair 90 min

|— Signal inspection 45 min

|— OHE inspection 60 min

Available block: 120 min

Possible bundle:

Track + Signal

OHE cannot fit → move to another opportunity

The system should maximize useful maintenance rather than blindly grouping tasks.

9. Optimization Engine

Build an optimization algorithm that minimizes:

- Train delays
- Maintenance backlog
- Safety risk
- Number of separate blocks
- Resource conflicts

- Unused block time

And maximizes:

- Maintenance completed
- High-priority work completed
- Multi-department coordination
- Asset availability
- Utilization of maintenance windows

A simple heuristic/weighted optimization approach is sufficient for the first version.

Architecture should allow replacing it later with:

- Genetic Algorithm
- MILP
- Constraint Programming
- OR-Tools
- Reinforcement Learning

Do not overcomplicate the first implementation.

10. What-If Simulation

Add a simulation feature.

Planner should be able to ask:

Scenario 1

"What happens if I cancel this block?"

Show:

- Additional train impact
- Maintenance backlog increase
- Risk increase
- New recommended schedule

Scenario 2

"What happens if I extend this block by 30 minutes?"

Show:

- Additional tasks completed

- Additional train impact
- Resource requirements
- Whether extension is recommended

Scenario 3

"What happens if a critical defect appears tomorrow?"

Show:

- Where the task can be inserted
- Which low-priority task should move
- Resulting schedule

Use a clear comparison:

CURRENT PLAN

Train delay: 8 min

Tasks completed: 7

Risk: 18

ALTERNATIVE PLAN

Train delay: 10 min

Tasks completed: 11

Risk: 12

Recommendation:

ALTERNATIVE PLAN

11. Real-Time Adaptive Replanning

The system should simulate changing railway conditions.

Examples:

- New critical defect
- Train delay
- Goods train added
- Crew becomes unavailable
- Machine failure
- Weather disruption
- Block cancelled

When an event occurs:

1. Detect affected maintenance plans
2. Identify impacted tasks
3. Recalculate available opportunities
4. Move flexible/low-priority work
5. Preserve safety-critical work
6. Generate updated plan
7. Show what changed

Example:

 PLAN CHANGE DETECTED

Reason:

Goods train added to Section B at 02:20

Previous:

02:00-04:00 maintenance block

New recommendation:

02:40-04:00

Moved:

Signal inspection → next opportunity

Retained:

Critical track repair

12. Weekly Planning

Create a weekly planning screen.

Display:

- Date
- Section
- Block/opportunity
- Start time
- End time
- Departments
- Tasks
- Priority
- Train impact

- Status

Use a Gantt-style timeline.

Example:

MONDAY			
Section A		02:00 — 04:00	
		Track + OHE	
Section B		01:30 — 03:00	
		S&T	

Allow planner to approve/reject/edit recommendations.

13. Monthly Planning

Monthly view should be higher level.

Show:

- Maintenance backlog
- Planned work
- High-risk assets
- Section-wise maintenance
- Department workload
- Expected block demand
- Planned vs available capacity

The monthly plan should not be as detailed as the weekly plan.

Weekly = execution planning.

Monthly = capacity and strategic planning.

14. Digital Twin / Railway Visualization

Create a visual railway corridor.

Example:

```
[Station A]
  |
Section A
  |
[Station B]
  |
Section B
  |
[Station C]
```

Clicking a section should show:

Section A

Track Health: 72% OHE Health: 84% Signal Health: 91%

Traffic: HIGH

Open Tasks: 8

Critical Tasks: 2

Next Opportunity: 02:00–04:00

Recommended: Track + OHE + Inspection

The visualization does not need to be a geographically accurate railway map. A schematic digital twin is sufficient for the prototype.

15. Dashboard KPIs

Main dashboard should show:

Maintenance KPIs

- Tasks completed
- Critical tasks completed
- Backlog
- Asset availability

Operations KPIs

- Number of blocks
- Average train delay

- Train delay avoided
- Block utilization

Coordination KPIs

- Combined blocks
- Departments coordinated
- Separate blocks avoided

Efficiency KPIs

- Maintenance hours saved
- Unused block time
- Resource utilization

Example:

Maintenance Tasks	126
Critical Tasks	18
Combined Blocks	34
Separate Blocks Avoided	21
Train Delay Reduced	28%
Block Utilization	87%
Asset Availability	94%

16. Explainable Recommendation

Every AI recommendation must have a "Why?" button.

Example:

Recommended Block

Section B-C 02:10-03:50

Why?

✓ Low train traffic ✓ 3 compatible maintenance tasks ✓ 2 departments can work simultaneously ✓ Required crew available ✓ Machine available ✓ Critical track defect nearby ✓ Expected train impact only 5 minutes

This is important because railway planners should be able to understand and trust the recommendation.

17. Before vs After Comparison

Create a screen comparing the existing/manual approach with the proposed intelligent approach.

Example:

Metric	Existing	Proposed
Blocks	42	29
Combined blocks	5	18
Train delay	185 min	112 min
Maintenance completed	76	94
Critical tasks pending	11	3
Block utilization	61%	88%

Use simulated data and clearly label it as **prototype/synthetic data**.

18. Alerts

Add alerts for:

- Critical defect
- Overdue maintenance
- Block conflict
- Resource conflict
- Train conflict
- Plan becoming infeasible
- New emergency maintenance requirement

Example:

 Critical track defect detected in Section B-C. Recommended action within 12 hours.

19. Suggested Technology Stack

Use a practical stack.

Frontend

- React
- TypeScript
- Tailwind CSS
- Recharts
- Leaflet or a custom SVG railway visualization
- Gantt/timeline component if useful

Backend

- Python
- FastAPI

AI/Optimization

- Python
- Pandas
- Scikit-learn
- OR-Tools or custom heuristic optimization

Database

- PostgreSQL

For a hackathon prototype, SQLite can be used initially if faster.

20. AI/ML Requirements

Do not claim that an LLM is responsible for railway safety decisions.

Use AI/ML for:

- Risk prediction
- Task prioritization
- Delay prediction
- Opportunity scoring
- Maintenance bundling
- Schedule optimization

Use deterministic constraints for:

- Safety restrictions
- Train conflicts
- Resource conflicts
- Block duration
- Mandatory maintenance requirements

The system should be:

AI-assisted + constraint-based + explainable

rather than a completely autonomous black-box system.

21. Demo Flow

The final demo should tell a story.

Step 1

Show railway corridor.

Step 2

Show incoming maintenance data:

TMS + SMMS + TDMS.

Step 3

Show 15–20 maintenance tasks.

Step 4

AI calculates priorities.

Step 5

System finds available railway windows from train data.

Step 6

System identifies:

"Section B has a 2-hour maintenance opportunity."

Step 7

System bundles Track + S&T + OHE work.

Step 8

Optimizer creates the weekly plan.

Step 9

Show before vs after metrics.

Step 10

Trigger an unexpected event:

"Goods train delayed and occupies Section B."

Step 11

System automatically replans.

Step 12

Show updated plan and explain why the change happened.

This should be the main hackathon demonstration.

22. Most Important Product Principle

Do NOT build only:

"A dashboard that displays maintenance blocks."

Build:

"A decision-support and optimization engine that continuously converts railway operating opportunities into the safest and most valuable maintenance plan."

The dashboard is only the interface.

The real product is the intelligence underneath.

23. MVP Priority

Build in this order:

P0 — Must Have

- Synthetic railway data
- Maintenance task database
- Train schedule
- Risk scoring
- Maintenance opportunity detection
- Multi-department task bundling
- Block optimization
- Weekly plan
- Dashboard
- Explainable recommendations

P1 — Important

- Monthly planning
- What-if simulation
- Before/after comparison
- Resource constraints
- Digital twin visualization

P2 — Differentiators

- Real-time replanning
- Emergency task insertion
- Delay prediction
- Advanced ML
- Advanced optimization

Do not spend most of the development time on UI before the optimization engine works.

24. Final One-Line Product Definition

Build an explainable AI-powered Railway Maintenance Orchestrator that finds the best maintenance opportunities, bundles compatible work across departments, optimizes train impact and resources, and continuously replans when railway conditions change.